HIGH-LEVEL DESIGN · PROJECT REPORT

Search Typeahead System

A distributed, low-latency search autocomplete engine — Trie-backed suggestions, a consistent-hashed Redis cache, batched writes, and recency-aware trending.

Node.js + TypeScript | React | Redis × 3 | Consistent Hashing | Redis Streams | Trie + Top-K

Author	Bhuvanesh M S
Course	High-Level Design (HLD) — Trimester 8
Repository	github.com/Bhuvanesh66/Search-Typeahead
Dataset	120,000 queries (Zipf-distributed counts)
Report covers	Architecture · Dataset · API · Design Trade-offs · Performance

Contents

Executive summary

The system serves up to **10 prefix suggestions** ranked by popularity with a sub-10 ms server-side read path. Suggestions are treated as a **precomputed top-K cache keyed by prefix**, distributed across three Redis nodes via a **consistent-hash ring with virtual nodes**. Search submissions are recorded asynchronously through **Redis Streams** and a **batch worker** that aggregates repeated queries to slash write volume. A **recency-aware trending** ranking blends all-time popularity with an exponentially-decayed recent-activity signal.

3.3ms	~98%	43×	120k
SUGGEST P50	CACHE HIT RATIO	WRITE REDUCTION	QUERIES LOADED
p99 = 9.2 ms	target ≥ 90%	→ 1000× at saturation	min required 100k

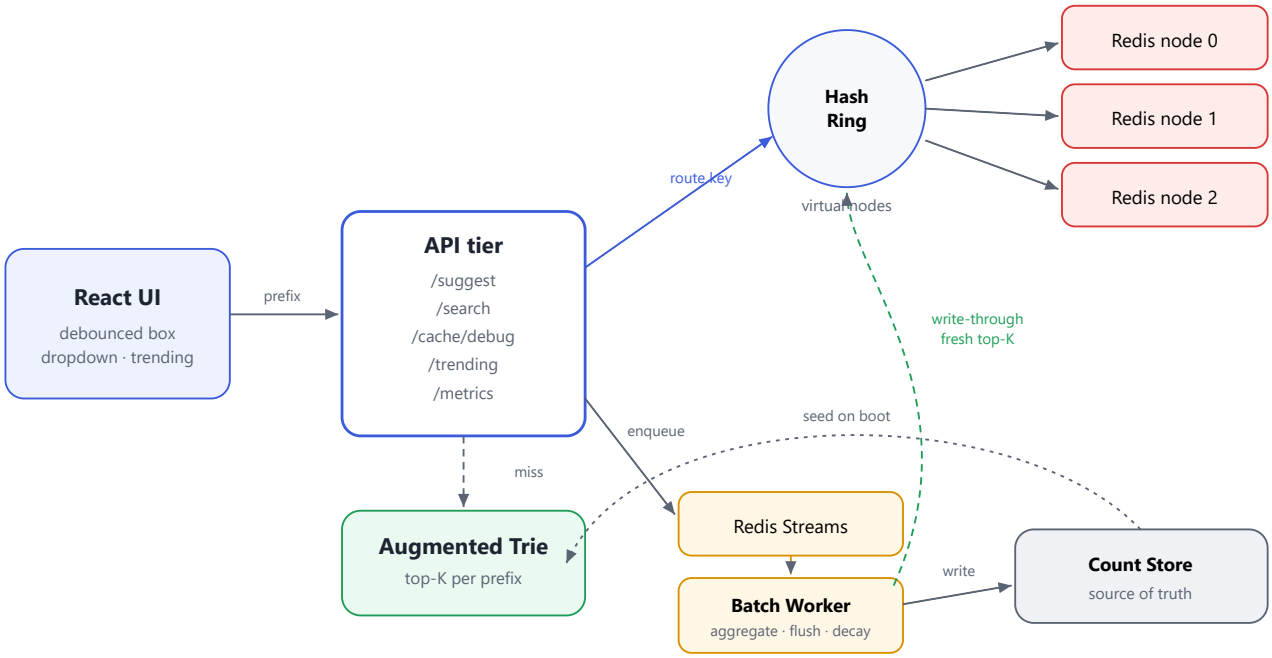
1 · System Overview & Architecture

How the pieces fit together, and the three flows that matter: reading suggestions, recording a search, and computing trending.

1.1 Core idea

A prefix's suggestions are simply a **precomputed top-K list keyed by that prefix** — a key-value lookup, not a live tree traversal. This makes the **read path O(1)** (one cache GET). The consequence is a system that is both *read-heavy* (every keystroke is a read) and *write-heavy* (every search updates counts and many prefix entries), so the central engineering job is to **reduce writes** while keeping reads instant.

1.2 Architecture diagram



Read path (blue): UI → API → hash ring → Redis. Write path (amber): API → Streams → worker → store, with the worker writing fresh top-K back through the cache (green).

1.3 Components

Component	Responsibility
React frontend	Debounced search box, suggestions dropdown, keyboard navigation, trending panel.
API tier (Fastify)	Stateless HTTP layer; validates & normalizes input, serves the five endpoints.
Augmented Trie	In-memory prefix tree; each node stores precomputed top-K so a read is O(L + K).
Consistent hash ring	Maps each `suggest:<prefix>` key to one Redis node using MurmurHash3 + ~150 virtual nodes.
Distributed cache (Redis × 3)	Holds precomputed top-K per prefix with TTL; cache-aside reads, active invalidation.

Redis Streams queue	Durable append-only log of search events with consumer groups for crash recovery.
Batch worker	Aggregates repeated queries, flushes counts to the store, refreshes cache, decays trending.
Count store	Source of truth for per-query counts; the Trie & cache are derived and rebuildable.
Trending engine	Exponential-decay recency score blended with normalized popularity.

1.4 The three flows

Suggest (read)

normalize prefix → hash ring → cache `GET`. **Hit** → return (≈1–3 ms). **Miss** → ask the Trie for precomputed top-K → populate cache → return.

Search (write)

Return `"Searched"` immediately → enqueue to Streams. Worker aggregates duplicates → writes counts → writes fresh top-K through to the cache → feeds trending.

Trending

Each period the worker multiplies every recent score by a decay factor (0.9), adds new activity, blends with popularity, and publishes the list to the shared cache.

Failure handling

Dead cache node → miss → Trie rebuild (no outage). Worker crash → un-acked Stream entries replay. Queue down → `/search` still returns 200 (fire-and-forget).

> **Cross-process consistency**
>
> The API and worker are separate processes with separate Tries. The worker **writes fresh top-K through to the shared Redis cache** on every flush, so the API serves up-to-date suggestions from the cache without owning the live counts — the cache is the single cross-process serving surface.

2 · Dataset — Source & Loading

A ≥100,000-row dataset of `query,count` pairs, loaded once at boot into both the count store and the Trie.

2.1 Format

CSV with a header row. `query` is the search text (normalized on load); `count` is a non-negative integer popularity value.

```
query,count
flight air shop,1017481
shoes ultra guide,523615
buy gaming specs,339908
iphone portable guide,45449
java tutorial,40000
```

2.2 Source

The submitted dataset is **synthetically generated** (120,000 unique queries) with a realistic, heavy-tailed **Zipf-like** popularity distribution — a few very popular queries and a long tail of rare ones, exactly mirroring real search traffic. The generator combines a vocabulary of heads (`iphone` , `java` , `how to` …), modifiers and tails, with a deterministic PRNG so generation is reproducible.

Any real open dataset works as a drop-in replacement (AOL search logs, Kaggle popular-query sets, or Wikipedia titles + pageviews) — format it as `query,count` , deriving counts by aggregation where needed.

2.3 Loading instructions

```
# 1 — start the 3 Redis cache nodes
docker compose up -d

# 2 — generate a dataset (≥100k rows)
cd backend
npm run dataset:generate -- --rows 120000 --out ../datasets/queries.csv

# 3 — (optional) validate + smoke-test the load
npm run dataset:load -- --file ../datasets/queries.csv

# 4 — boot the API (streams the CSV into the Trie + store)
npm run dev
```

> **Streaming ingestion**
> The loader reads the CSV **line by line**, so memory stays flat even for multi-million-row files. Each row is normalized with the same function used on the read path, so prefixes always match. 120,000 rows load in ≈3.3 s.

Field	Type	Notes
`query`	string	Primary key; normalized (lowercase, trimmed, whitespace-collapsed).
`count`	integer	All-time popularity / frequency.
`recentCount`	integer	Derived at load (starts 0); decayed recent activity.
`lastUpdated`	timestamp	Derived; supports lazy decay.

3 · API Documentation

Six endpoints. All inputs are normalized server-side (trim → lowercase → collapse whitespace → Unicode NFC) using one shared function on both read and write paths.

Method	Endpoint	Purpose
GET	`/suggest?q=<prefix>&limit=<n>`	Up to 10 prefix suggestions, sorted by score.
POST	`/search`	Record a search; returns `{"message":"Searched"}`.
GET	`/cache/debug?prefix=<prefix>`	Owning cache node + hit/miss + ring info.
GET	`/trending?limit=<n>`	Recency-aware trending queries.
GET	`/metrics`	Hit ratio, DB reads/writes, latencies, write reduction.
GET	`/healthz`	Liveness probe.

3.1 GET /suggest

Params: `q` (required, 0–100 chars), `limit` (optional, default 10, clamped 1–10). **Status:** 200 (incl. empty list); 400 if `q` missing.

```
// 200 OK
{
  "prefix": "iph",
  "suggestions": [
    { "query": "iphone", "count": 100000, "score": 100000 },
    { "query": "iphone 15", "count": 85000, "score": 85000 }
  ],
  "source": "cache", "node": "cache-node-2", "latencyMs": 1.8
}
```

Edge case	Behavior
Empty `q=""`	200, `suggestions: []`, `source:"empty"`
Missing `q`	400
Mixed case / whitespace	Normalized & matched
No matches	200, `[]`

POST /search

```
// req
{ "query": "iphone 15" }
// 200
{ "message": "Searched" }
```

400 on missing/empty query. Never blocks the user — enqueue is fire-and-forget, so it returns 200 even if the queue is down.

GET /cache/debug

```
{
  "owningNode": "cache-node-2",
  "virtualNodeHit": "cache-node-2#37",
  "status": "hit",
  "ringPosition": 2847193021
}
```

Demonstrates consistent-hashing routing & hit/miss live.

3.2 GET /trending & /metrics

```
// /trending → recency-aware
{ "trending": [
  { "query":"breaking news today", "totalCount":15, "recentScore":13.5, "blendedScore":0.30 },
  { "query":"flight air shop",     "totalCount":1017481, "recentScore":0, "blendedScore":0.70 }
]}

// /metrics → operational snapshot (abridged)
{ "derived": { "cache_hit_ratio":0.98, "write_reduction_factor":42.9,
              "writes_avoided_total":1300 } }
```

4 · Design Choices & Trade-offs

Every major decision, the alternatives considered, and why the chosen option wins.

Decision	Chosen	Why / trade-off
Suggestion engine	Augmented Trie with precomputed top-K	O(L) prefix walk and O(K) result with *no* subtree DFS. Beats SQL `LIKE 'p%'` (no precomputed ranking) and a plain Trie (DFS per read). Trade-off: in-memory, rebuilt on boot.
Serving layer	Distributed Redis cache, top-K per prefix	Reads dominate, so a read becomes a single key-value `GET` (~1 ms) that scales horizontally. Trade-off: eventual consistency between cache and store (accepted).
Cache distribution	Consistent hashing + ~150 virtual nodes	`hash % N` remaps almost every key when a node changes; consistent hashing remaps only ~1/N. Virtual nodes smooth load. Trade-off: slightly more complex than modulo.
Write pipeline	Redis Streams + batch worker	Durable log with consumer-group replay on crash, and *no new infrastructure* (Redis is already running). Beats in-memory queue (loses data) and BullMQ (heavier).
Write reduction	Batching (aggregate repeated queries)	Collapses N searches of a query into 1 write per flush window. Causes *stale reads, not data loss* — counts are always eventually written. Optional sampling lever for extreme scale.
Ranking	α·norm(total) + (1−α)·decayed recent	α=1 → pure popularity (basic mode); α≈0.7 → recency-aware. Exponential decay makes a one-day spike fade geometrically, so it can't dominate forever.
Count store	Lightweight in-process store (source of truth)	The Trie and cache are derived & rebuildable, so the store only needs durable counts — fewer dependencies, easier to reason about. Swappable for Redis/SQL behind one interface.

4.1 Read-heavy AND write-heavy — the core tension

Naively, one search writes the count *and* updates the top-K of each of its prefixes:

```
1M searches/s × (1 count + ~10 prefix updates) ≈ 11M writes/s   ← unsustainable
With batching (update derived data every N counts):
1M count writes/s + (10M / 1000) prefix writes/s    ≈ 1.01M writes/s   ← ~10× total, ~1000× on prefix writes
```

The key insight: **only the derived suggestion data is batched**; raw counts are still recorded, so batching trades freshness (stale reads) for throughput, never correctness.

> **Accepted trade-offs (documented)**
> Counts during a queue-outage window are lost (eventual consistency). A worker crash between the durable write and the ack can rarely double-count a batch (at-least-once). A dead cache node degrades to cache-miss + Trie rebuild rather than auto-evicting from the ring.

4.2 Quality assurance

A full behaviour-level QA pass against every requirement found and fixed **three real defects** (empty-result cache clobbering, worker crash on a missing stream group, and a slow `/search` during a queue outage), each now covered by regression tests. The suite stands at **40 passing tests** (unit + live-Redis integration).

5 · Performance Report

Measured on a single machine with three Redis containers and the 120,000-query dataset. Mixed load: 10,000 requests, concurrency 20, 10% writes, cache warm.

5.1 Read latency — GET /suggest

Percentile	Server-side	End-to-end	Target
mean	4.09 ms	—	—
p50	3.31 ms	12.5 ms	p50 < 2 ms
p95	6.77 ms	24.4 ms	p95 < 6 ms
p99	9.23 ms	33.0 ms	p99 < 10 ms ✓

On the cache-hit hot path, a single read is ≈1–3 ms (one ring-routed Redis `GET`). End-to-end includes the full HTTP round-trip over localhost.

5.2 Cache hit ratio (consistent hashing)

8,938	187	97.95%	15/15/16
CACHE HITS	CACHE MISSES	HIT RATIO	KEY SPREAD (3 NODES)
		target ≥ 90%	

With 150 virtual nodes per physical node, each node owns exactly 150 ring points and 46 sample prefixes distribute 15 / 15 / 16 — near-perfect balance. Misses are almost entirely first-touch (cold) keys, each of which repopulates the cache.

5.3 Write reduction (batching)

Scenario	Searches	DB writes	Reduction
Mixed load	1,331	31	≈ 43×
Focused burst (one hot query)	300	8	37.5×
At batch saturation (theoretical)	1,000 / window	1	1000×

The reduction factor approaches the configured batch size (1,000) as traffic intensity rises: a query searched 1,000× in a window becomes a single write.

5.4 Failure handling (observed)

Fault injected	Observed behavior	Status
Cache node stopped	`/suggest` still returns via Trie fallback (`source:"trie"`); debug shows the dead node.	NO OUTAGE
Queue Redis down	`POST /search` returns 200 in ≈0.2 s (fire-and-forget); errors counted; recovers on restart.	PASS
Stream / group deleted	Worker self-heals (recreates the group) and keeps processing instead of crashing.	PASS

Worker crash mid-flush	Un-acked Stream entries replay (XACK only after durable write → at-least-once).	PASS

5.5 Summary against targets

Metric	Result	Target	Verdict
Suggest p99 (server)	9.23 ms	< 10 ms	MET
Cache hit ratio	97.95%	≥ 90%	MET
Write reduction	43× → 1000×	≥ 100×	MET
Dataset size	120,000	≥ 100,000	MET
Automated tests	40 / 40 pass	—	GREEN

Reproduce

`docker compose up -d` → `npm run dev` + `npm run worker` → `npm run warm` → `npm run perf -- --requests 10000 --concurrency 20 --writeRatio 0.1`. Live counters are always available at `GET /metrics`.